

Practical 7

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Process	Allocation				MAX			
	Printer	ROM	Hard Disk	RAM	Printer	ROM	Hard Disk	RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

Write a program to:

- Calculate current need of resources by each process.
- Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

```
1) #include <stdio.h>
```

```
#define P 5 // Number of Processes
```

```
#define R 4 // Number of Resources
```

```
int main() {
```

```
    int alloc[P][R], max[P][R], need[P][R];
```

```
    int i, j;
```

```

printf("Enter Allocation Matrix (5x4):\n");
for(i = 0; i < P; i++) {
    for(j = 0; j < R; j++) {
        scanf("%d", &alloc[i][j]);
    }
}
printf("Enter Max Matrix (5x4):\n");
for(i = 0; i < P; i++) {
    for(j = 0; j < R; j++) {
        scanf("%d", &max[i][j]);
    }
}
for(i = 0; i < P; i++) {
    for(j = 0; j < R; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
printf("\nNeed Matrix is:\n");
for(i = 0; i < P; i++) {
    for(j = 0; j < R; j++) {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}
return 0;
}

```

```
ASUS@LAPTOP-LCU90K2P MSYS ~  
$ nano bankers.c
```

```
ASUS@LAPTOP-LCU90K2P MSYS ~  
$ gcc bankers.c -o bankers
```

```
ASUS@LAPTOP-LCU90K2P MSYS ~  
$ ./bankers  
Enter Allocation Matrix (5x4):
```

```
0 0 1 4  
0 6 3 2  
0 0 1 2  
1 0 0 0  
1 3 5 4
```

```
Enter Max Matrix (5x4):
```

```
0 6 5 6  
0 6 5 2  
0 0 1 2  
1 7 5 0  
2 3 5 6
```

```
Need Matrix is:
```

```
0 6 4 2  
0 0 2 0  
0 0 0 0  
0 7 5 0  
1 0 0 2
```

```
ASUS@LAPTOP-LCU90K2P MSYS ~  
$
```

```

2) #include <stdio.h>

#define P 5

#define R 4

int main() {

    int alloc[P][R], max[P][R], need[P][R];

    int avail[R];

    int finish[P] = {0};

    int safeSeq[P];

    int i, j, count = 0;

    printf("Enter Allocation Matrix:\n");

    for(i=0;i<P;i++)

        for(j=0;j<R;j++)

            scanf("%d",&alloc[i][j]);

    printf("Enter Max Matrix:\n");

    for(i=0;i<P;i++)

        for(j=0;j<R;j++)

            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");

    for(i=0;i<R;i++)

        scanf("%d",&avail[i]);

    for(i=0;i<P;i++)

        for(j=0;j<R;j++)

            need[i][j] = max[i][j] - alloc[i][j];

    while(count < P) {

        int found = 0;

        for(i=0;i<P;i++) {

            if(finish[i] == 0) {

```

```

int flag = 0;
for(j=0;j<R;j++) {
    if(need[i][j] > avail[j]) {
        flag = 1;
        break;
    }
}

if(flag == 0) {
    for(j=0;j<R;j++)
        avail[j] += alloc[i][j];

    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
}

}

if(found == 0) {
    printf("System is NOT in safe state\n");
    return 0;
}

}

printf("System is in SAFE state\nSafe sequence is:\n");
for(i=0;i<P;i++)

```

```
    printf("P%d ", safeSeq[i]);

    return 0;
}
```

```
ASUS@LAPTOP-LCU90K2P MSYS ~
$ ./bankers
Enter Allocation Matrix:
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4
Enter Max Matrix:
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6
Enter Available Resources:
1 6 2 0
System is in SAFE state
Safe sequence is:
P1 P2 P3 P4 P0
ASUS@LAPTOP-LCU90K2P MSYS ~
$
```